

- functional data structure,
 - recursion over lists
(*continued*)
 - generalizing to higher-order functions 38–41
 - useful methods for 43–44
- trees 44–47
- functional programming
 - advantages of
 - functional solution 6–9
 - program with side effects 4–6
 - effects in 209
 - functions, deriving implementation from types 25–28
 - imperative programming vs. 89
 - importance of laws 113
 - pure functions 9–10
 - referential transparency 10–12
- Scala
 - example program 15–17
 - higher-order functions 19, 21–22
 - loops as functions 20–21
 - modules 18–19
 - namespaces 18–19
 - objects 18–19
 - polymorphic
 - functions 22–25
 - running programs 17–18
- functions
 - anonymous 39–40
 - closing over environment 26
 - higher-order
 - example of 21–22
 - overview 19
 - lifting 56–60
 - loop as 20–21
 - polymorphic
 - calling HOFs 24–25
 - example of 23–24
 - overview 22
 - strict and non-strict 65–68
 - total function 52
 - as values 25
 - variadic 34
- functors
 - applicative functors
 - advantages of 211–214
 - applicative trait 206–208
 - laws 214–218
 - monads vs. 208–211

- turning monoid into applicative 220–221
- generalizing map function 187–189
- laws 189–190
- traversable functors
 - combining 223–224
 - fusion of 224
 - nesting 224–225
 - overview 218–219
 - with state 221–223
- fusion 224, 275

G

- generalizing to higher-order functions 38–41
- generator 130–132
- generic function 22–23
- guarded recursion 75

H

- hashCode method 25
- higher-kinded types 183
- HOFs (higher-order functions)
 - example of 21–22
 - generalizing to 38–41
 - overview 19
 - property-based testing for 142–144
 - type inference for 37–38

I

- I/O (input/output)
 - asynchronous 241–242, 247–250
 - console I/O 243–246
 - creating type for 231–232, 235–236, 250
 - effects vs. side effects 250–251
 - free monads 242–243
 - handling input effects 232–235
 - non-blocking 247–250
 - pure interpreters 246–247
 - streams 251–253
 - applications for 292–293
 - extensible process type 278–292
 - imperative I/O vs. 268–271
 - stream transducers 271–278

- identity element 175, 197
- identity laws 197–198
- identity monad 199–200
- immutable field 7
- imperative I/O 268–271
- imperative programming 268
 - functional programming vs. 89
 - state and 88–91
- implicit conversion 107
- import 18
- impure function 17
- impurity 4
- incremental I/O 269
- incremental implementation 72
- infinite stream 73–77
- infix syntax 19, 107
- inner function 20
- input effects 232–235
- intensional equality 266
- intensionality 266
- IO monad 229

J

- JSON parser
 - example of 159–160
 - format and 158–159
 - overview 158

K

- Kleisli composition 196–197

L

- labeling parsers 167–168
- laws 96, 147
 - of applicative functors
 - associativity 215–216
 - left and right identity 214–215
 - naturality 216–218
 - of forking 112
 - of functors 189–190
 - of generators 144
 - of mapping 110–112
 - of monads
 - associative law 194–196
 - identity laws 197–198
 - Kleisli composition 196–197
 - proving associative law 196–197